Resolva cada exercício em folhas separadas

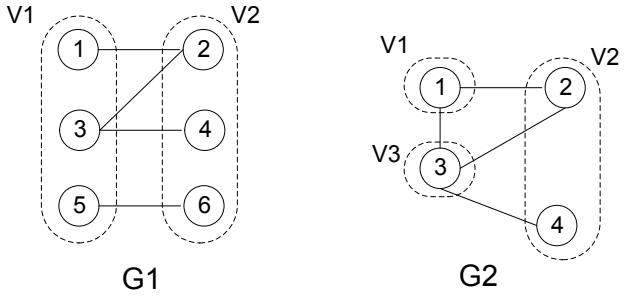
1. Dada uma lista de inteiros não repetidos pretende-se obter para um dado elemento os seus **K-vizinhos mais próximos**, ou seja, os elementos da lista com a menor diferença absoluta entre o elemento dado.

Como exemplo considere a seguinte lista L = { 2, 9, 7, 5, 10, 15, 6, 12, 3} e K=3. Os 3 vizinhos mais próximos do elemento 5 são {6, 3, 7}.

Elabore um método eficiente que recebe um conjunto não ordenado de números inteiros representado numa lista ligada, um elemento, o número de vizinhos – K e devolve a lista com os K-vizinhos mais próximos do elemento.

```
DoublyLinkedList<Integer> Kneighbors(DoublyLinkedList<Integer> list, Integer elem, Integer k)
```

2. Um grafo G=(V,E) não dirigido diz-se bipartido se é possível partir o conjunto V de vértices em dois subconjuntos V_1 e V_2 disjuntos de tal forma que quaisquer dois vértices adjacentes em G pertencem a subconjuntos diferentes. Por exemplo o grafo G_1 da figura é bipartido, enquanto que o grafo G_2 não é bipartido.



Usando a representação **map de adjacência** escreva um método que verifica se um grafo é bipartido e, em caso afirmativo, apresenta a possível partição dos vértices.

3. Considere o seguinte algoritmo:

```
Algorithm mistery (Node x){
      if (x == null) return;
      process(x);
      mistery(x.left);
      mistery(x.right);
 }
```

a) Explique o que faz o método acima apresentado.

b) Analise o método quanto à sua complexidade temporal. Justifique.

4. Adicione à classe TREE<E> um método que verifica se uma árvore binária de pesquisa é AVL.

5. Usando os métodos da classe HeapPriorityQueue<K,V> escreva um método eficiente que faz a junção ordenada de duas heaps num vector.